

AhamX Bodhi

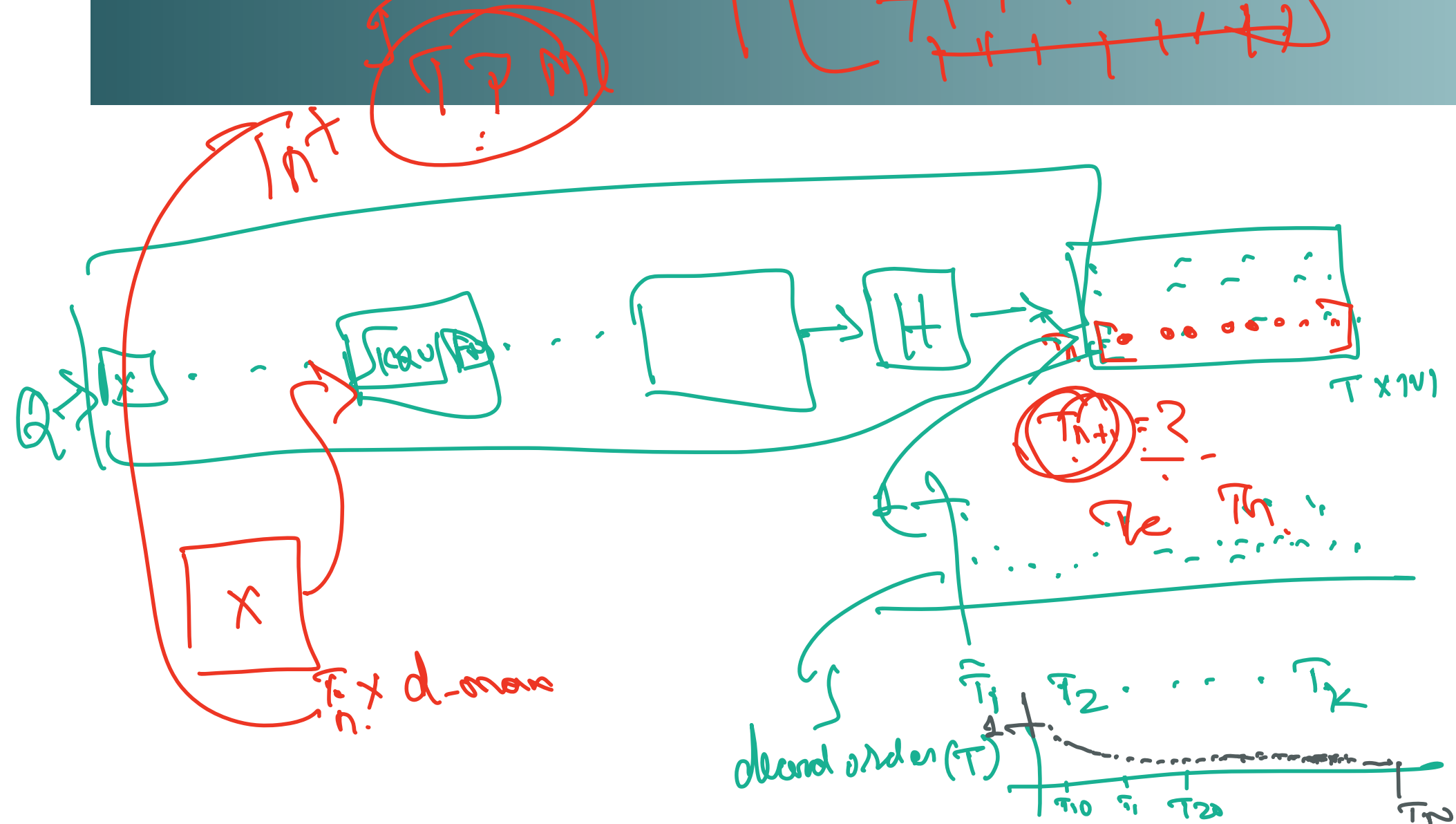
Where I Becomes Infinite

Temperature Parameter

Module 3.1: Working with LLM APIs

Prof. Sashikumaar Ganesan | IISc Bengaluru | 2026







Session Overview

Prerequisites

- How LLMs produce probability distributions over tokens
- The softmax function and logits
- Basics of LLM API calls (model, messages)
- Concept of next-token prediction

What You Will Learn

- The mathematical definition of temperature
- How temperature reshapes the softmax distribution
- Practical value ranges and their effects
- When to use high vs. low temperature
- Temperature interactions with top-p and top-k



The Problem: Controlling Output Randomness

Why We Need Temperature

At every step, an LLM outputs a probability distribution over tens of thousands of tokens. Without any adjustment, the raw model distribution may be too **peaked** (repetitive, conservative) or too **flat** (incoherent, random) for a given task. Temperature provides a single scalar knob to shift the distribution toward determinism or diversity.

Two Extremes Without Temperature Control

- **Always greedy (argmax):** Picks the highest-probability token every time. Output is deterministic but repetitive and uncreative
- **Uniform sampling:** Picks from all tokens equally. Output is maximally random and largely incoherent
- **Temperature:** Continuously interpolates between these extremes by scaling the logits before softmax



Temperature: Mathematical Definition

Temperature-Scaled Softmax

Standard softmax converts logits z_i to probabilities:

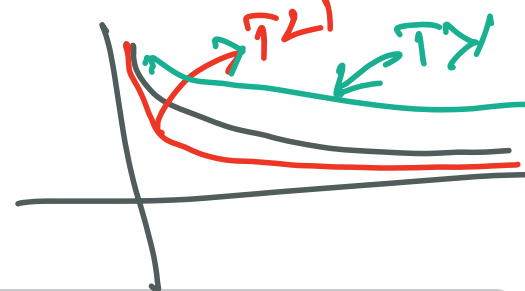
$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

With temperature $T > 0$, each logit is divided by T before softmax:

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $T = 1$: Standard softmax — raw model distribution (default)
- $T \rightarrow 0^+$: Distribution sharpens to a one-hot (greedy decoding)
- $T \rightarrow \infty$: Distribution flattens to uniform (pure random sampling)

Geometric Intuition: Low vs. High Temperature



Low Temperature ($T < 1$)

- Dividing by a small T **amplifies** differences between logits
- Tokens with high logits gain even higher probability
- Tokens with low logits are suppressed toward zero
- Result: sharp, concentrated distribution
- Analogy: a mountain sharpened to a spike

High Temperature ($T > 1$)

- Dividing by a large T **compresses** logit differences
- The gap between high- and low-scoring tokens shrinks
- Lower-probability tokens get a meaningful chance
- Result: flat, spread-out distribution
- Analogy: a mountain eroded to a gentle hill



Worked Example --- How Temperature Shifts Probabilities

Three Candidate Tokens, Three Temperatures

Suppose logits for the next token are: **Token A** $z = 3.0$, **Token B** $z = 1.5$, **Token C** $z = 0.5$

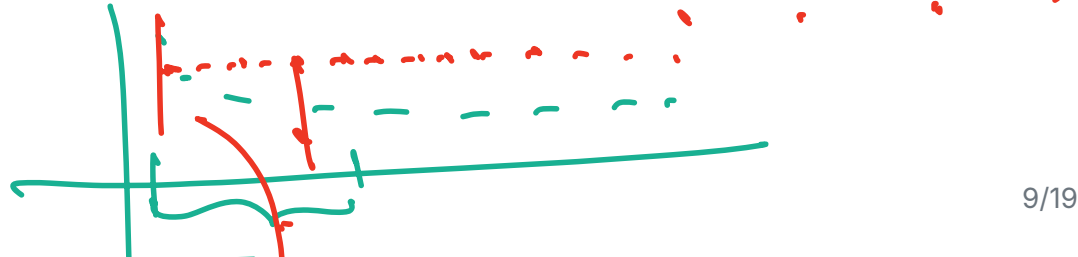
Token	$T = 0.5$	$T = 1.0$	$T = 2.0$
Token A ($z=3.0$)	0.952	0.825	0.596
Token B ($z=1.5$)	0.046	0.153	0.278
Token C ($z=0.5$)	0.002	0.022	0.127

At $T = 0.5$: Token A dominates completely. At $T = 2.0$: Token C has 12.7% chance vs. 0.2% — creative variation becomes possible.



Temperature Values --- Practical Reference Guide

Value	Label	Output Character	Best Use Cases
0.0	Deterministic	Identical on every call	Unit tests, exact extraction
0.1–0.3	Near-greedy	Very consistent, focused	Code generation, factual Q&A
0.4–0.6	Balanced	Slight variation, coherent	Summarisation, data extraction
0.7–0.9	Moderate creative	Varied, natural language	Chatbots, email drafting
1.0	Raw distribution	Model default diversity	General assistant interactions
<u>1.1–1.5</u>	High creative	Noticeable variation	Brainstorming, creative writing
1.6–2.0	Very high	Erratic, often incoherent	Experimental, avoid in production



Temperature and Reproducibility

Handwritten notes in green and red ink:

- Green: $T_{n+1} = ?$
- Red: 0.001
- Red: 0.001



Setting Temperature to 0

- $T = 0$ makes generation fully deterministic
- Same prompt always produces same output
- Equivalent to greedy argmax decoding
- Essential for: unit testing, regression tests, audit trails, and reproducible results in scientific or legal contexts

Reproducibility Caveats

- Even at $T = 0$, outputs may differ across **model versions** or when the provider updates serving infrastructure
- Floating-point non-determinism on different GPU hardware can introduce rare differences
- Use **seed** parameter (OpenAI) alongside $T = 0$ for maximum reproducibility guarantees
- Always pin the specific model version string



Temperature Interacts With Other Sampling Parameters

Order of Operations in Sampling

When multiple sampling parameters are set, they are applied sequentially: **(1)** Logits are divided by temperature to get $p_i(T)$. **(2)** Top-K filters to keep only the k highest-probability tokens. **(3)** Top-P (nucleus) then takes the smallest set summing to $\geq p$. **(4)** A token is sampled from the remaining distribution.

Practical Interaction Rules

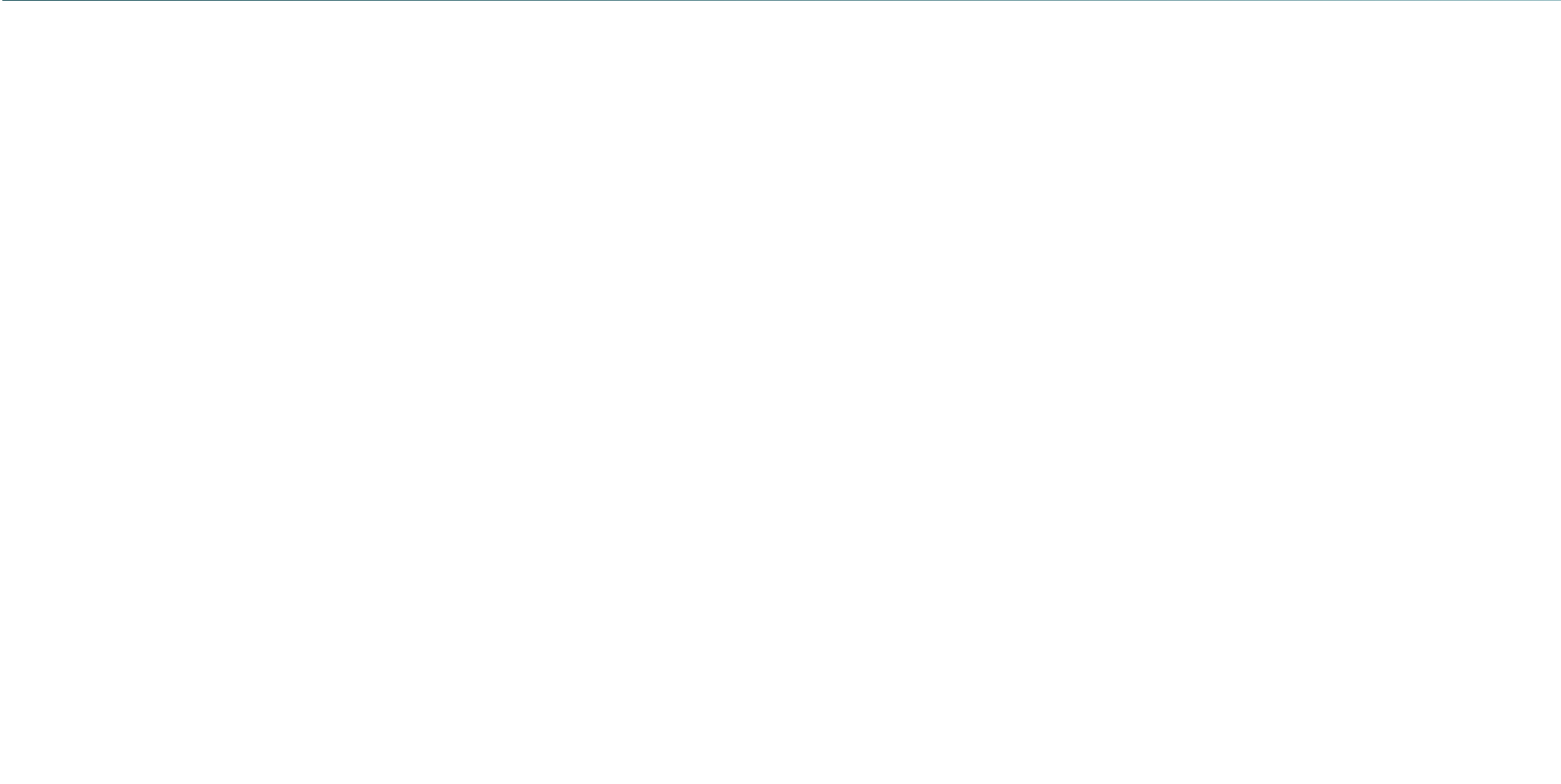
- Temperature affects the *shape* before top-k/top-p filtering
- High temperature + low top-p: diversity limited by nucleus cutoff
- Low temperature + high top-p: distribution already peaked, top-p barely constrains it
- **Best practice:** Tune temperature first, then apply top-p as a safety guard; avoid setting all three simultaneously
- OpenAI recommends: set temperature *or* top-p, not both



Common Misconceptions About Temperature

Myths vs. Reality

- **Myth: High temperature makes the model smarter.**
Reality: Temperature reshapes the distribution but adds no new knowledge. High temperature increases diversity, not accuracy.
- **Myth: Temperature = 0 always gives the best answer.**
Reality: Greedy decoding can fall into repetitive loops and misses valid alternative phrasings. Low temperature is best for structured tasks, not necessarily for quality.
- **Myth: Temperature affects only the last token.**
Reality: Temperature is applied at *every* generation step, shaping the entire output sequence cumulatively.
- **Myth: Temperature and randomness are the same concept.**
Reality: Temperature is a specific mathematical operation on logits. Randomness in LLMs also depends on top-p, top-k, and the seed.





GenAI Application: Code Generation

Low Temperature for Syntactic Correctness

- **GitHub Copilot:** Uses temperature ≈ 0.1 to ensure syntactically valid, idiomatic completions. At higher temperatures, code introduces random variable names, wrong API calls, or syntax errors
- **Cursor AI:** Similarly uses near-zero temperature for autocomplete; slightly higher (≈ 0.3) when generating entire function bodies where some variation in approach is acceptable
- **OpenAI Codex (legacy):** Default temperature was 0 for structured code generation and 0.5–0.8 for exploratory code brainstorming tasks
- **Amazon CodeWhisperer:** Low temperature with repetition penalty to prevent repeated boilerplate insertion

Code is syntactically unforgiving — a small random deviation can break compilation. Low temperature is the standard choice for all code generation tasks.



GenAI Application: Creative Writing and Ideation

Higher Temperature for Creative Diversity

- **Jasper AI (marketing copy):** Uses temperature 0.7–0.9 to generate varied ad headlines and taglines. Multiple calls to the same prompt produce distinct creative options for A/B testing
- **Character.ai:** Conversational character personas use moderate to high temperature (0.8–1.1) to produce more human-like, varied dialogue rather than formulaic responses
- **Sudowrite (fiction):** Implements multiple temperature presets. “Story Engine” mode uses high temperature for unpredictable plot suggestions; “Edit” mode uses low temperature for consistency
- **Brainstorming tools:** Set temperature to 1.0–1.3 deliberately to surface unusual word associations and non-obvious ideas

Creative tasks benefit from diversity; factual and structured tasks benefit from consistency.

GenAI Application: Customer Support Automation



Production Temperature Settings

- **FAQ answering:** $T = 0.1-0.3$ for consistent, accurate answers to policy questions
- **Ticket classification:** $T = 0$ for deterministic routing to correct department queue
- **Empathetic responses:** $T = 0.5-0.7$ for varied, natural-sounding acknowledgment phrases
- **Escalation decisions:** $T = 0$ for binary decisions — never randomise escalation logic

Why Not Use Temperature=1 Everywhere?

- Inconsistent advice creates customer confusion
- High-temperature policy answers may contradict company documentation on different calls
- Regulatory environments (banking, healthcare) require auditable, consistent outputs
- Temperature is a business risk knob, not just a technical parameter



GenAI Application: RAG and Information Extraction

Low Temperature Keeps Answers Grounded in Retrieved Context

- **Enterprise search (Glean, Notion AI):** Temperature 0.0–0.2 for document Q&A. The correct answer is *in* the retrieved context — temperature should not allow the model to wander away
- **Legal document review:** Near-zero temperature for contract clause extraction. A random paraphrase of a clause could have significant legal consequences
- **Medical information retrieval:** Low temperature (≤ 0.2) to prevent the model from “creatively” combining medical facts in ways not supported by the retrieved evidence
- **Structured data extraction (JSON output):** $T = 0$ or $T = 0.1$ when parsing structured fields from documents — schema compliance depends on deterministic token choices

Session Summary



Key Takeaways

1. **Temperature** divides all logits by a scalar T before the softmax, reshaping the probability distribution at every step
2. $T < 1$ sharpens the distribution (more deterministic); $T > 1$ flattens it (more diverse); $T = 1$ is the default
3. **Temperature does not add knowledge** — it only changes which tokens are likely to be sampled
4. Practical ranges: 0.0–0.3 for code/facts; 0.7–1.0 for chat; 1.0–1.5 for creative tasks
5. Temperature is applied *before* top-p and top-k; tune it first, then use nucleus/top-k as a safety constraint
6. $T = 0$ provides deterministic outputs essential for testing and reproducibility

AhamX Bodhi

Where I Becomes Infinite